O nível de arquitetura do conjunto de instrução

ISA

Assuntos das Aulas

- Entrada/Saída
- Propriedades do nível ISA
- Modelos de memória
- Registradores
- Instruções
- Tipos de dados
- Formatos de instrução
- Tipos de Endereçamento
- Tipos de instrução
- Instruções para movimento de dados
- Operações diádicas
- Operações monádicas
- Comparações e desvios condicionais
- Instruções de chamada de procedimento
- Controle de laço
- Fluxo de controle sequencial e desvios
- Exceções

Entrada/Saída

Nenhum outro grupo de instruções exibe tanta variedade entre máquinas quanto as instruções de E/S. Há três esquemas diferentes de E/S em uso corrente em computadores pessoais. São eles:

1. E/S programada com espera ocupada.

2. E/S por interrupção.

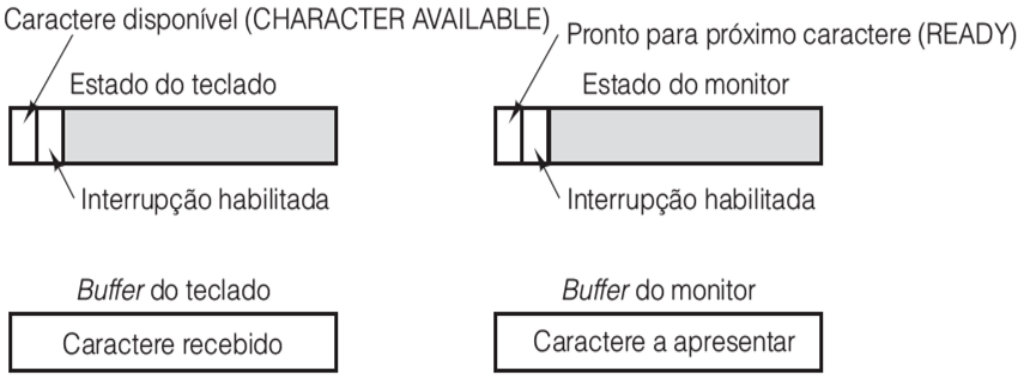
3. E/S por DMA.

Entrada/Saída

- O método de E/S mais simples possível é a **E/S programada**,
  - mais usada em microprocessadores de baixa tecnologia, por exemplo, em sistemas embarcados
  - ou em sistemas que têm de responder rapidamente a mudanças externas (sistemas de tempo real).
- Essas CPUs costumam ter uma única instrução de entrada e uma única instrução de saída.
- Cada uma das instruções seleciona um dos dispositivos de E/S.
- Um único caractere é transferido entre um registrador fixo no processador e o dispositivo de E/S selecionado.
- O processador deve executar uma sequência de instruções para todo e qualquer caractere lido ou escrito.

Entrada/Saída

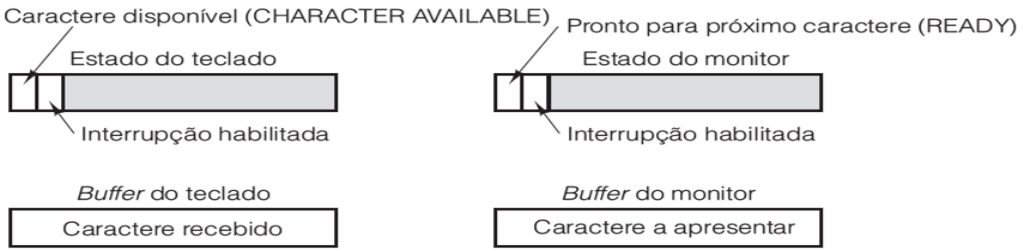
Figura 5.30 Registradores de dispositivo para um terminal simples.



- Como um exemplo desse método, considere um terminal com quatro registradores de 1 byte.
  - Dois registradores são usados para entrada, um de estado e um de dados,
  - e dois para saída, também um para estado um para dados.
  - Cada um tem um endereço exclusivo.
- Se a E/S usada for do tipo mapeada para a memória, todos os quatro registradores são parte do espaço de endereço da memória do computador e podem ser lidos e escritos usando instruções comuns.
- Caso contrário, são fornecidas instruções especiais de E/S, por exemplo, IN e OUT, para ler e escrever nos registradores.
- Em ambos os casos, a E/S é executada por transferência de dados e informação de estado entre a CPU e esses registradores.

Entrada/Saída

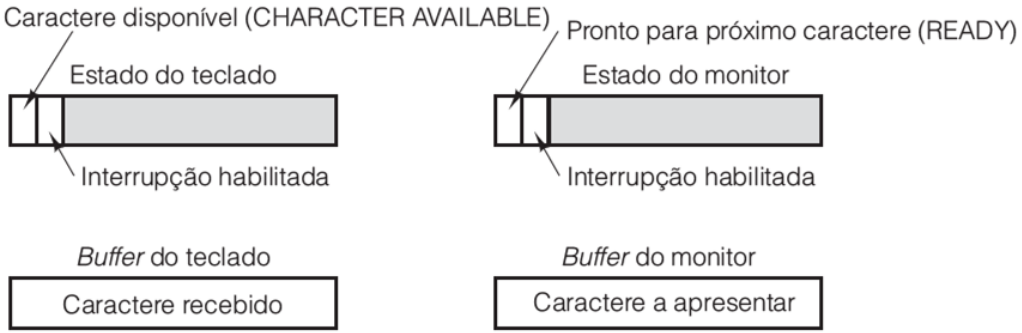
Figura 5.30 Registradores de dispositivo para um terminal simples.



- O registrador de estado do teclado tem 2 bits que são usados e 6 bits que não o são.
  - O bit da extrema esquerda (7) é marcado em 1 pelo hardware sempre que um caractere chegar.
- Se o software tiver marcado antes o bit 6, é gerada uma interrupção; caso contrário, ela não é gerada.
- Quando usa E/S programada para obter entrada, a CPU normalmente se encontra em um laço estreito que lê repetidas vezes o registrador de estado do teclado à espera do bit 7 para então continuar.
- Quando isso acontece, o software lê o registrador de buffer do teclado para obter o caractere.
- Ao ler o registrador de dados do teclado, o registrador faz com que o bit CHARACTER AVAILABLE (caractere disponível, bit 7) seja retornado para 0.

Entrada/Saída

Figura 5.30 Registradores de dispositivo para um terminal simples.



- A saída funciona de modo semelhante.
- Para escrever um caractere na tela, o software primeiro lê o registrador de estado do monitor para ver se o bit READY (pronto para próximo caractere) é 1.
- Se não for, executa o laço até que o bit vá para 1, o que indica que o dispositivo está pronto para aceitar um caractere.
- Quando o terminal está pronto, o software escreve um caractere para o registrador de buffer do monitor, o que faz com que ele seja transmitido para a tela, e também com que o dispositivo libere o bit READY no registrador de estado do monitor.
- Quando o caractere for apresentado na tela e o terminal estiver pronto para tratar o próximo caractere, o controlador automaticamente ajusta o bit READY para 1 novamente.

Entrada/Saída

- Como um exemplo de E/S programada, considere o procedimento Java ao lado.
- Esse procedimento é chamado com dois parâmetros: um vetor de caracteres para saída e o número de caracteres presentes no vetor, até 1 K.
- O corpo do procedimento é um laço que envia caracteres para a saída um por vez. Para cada caractere, primeiro a CPU tem de esperar até que o dispositivo esteja pronto, e então envia o caractere.
- Os procedimentos in e out seriam típicas rotinas de linguagem de montagem para ler e escrever os registradores do dispositivo especificados pelo primeiro parâmetro de ou para a variável especificada como o segundo parâmetro.

Figura 5.31 Exemplo de E/S programada.

```
public static void output_buffer(char buf[ ], int count) {
    // Produza um bloco de dados para o dispositivo
    int status, i, ready;
    for (i = 0; i < count; i++) {
        do {
            status = in(display_status_reg);      // obtenha estado
            ready = (status >> 7) & 0x01;         // isole o bit de pronto
        } while (ready != 1);
        out(display_buffer_reg, buf[i]);
    }
}
```

Entrada/Saída

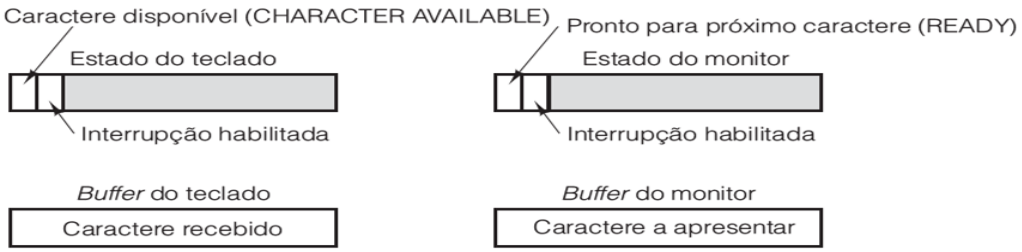
- A divisão por 128, implícita pelo deslocamento, se livra dos 7 bits de ordem baixa deixando o bit READY no bit 0.
- A principal desvantagem da E/S programada é que a CPU gasta grande parte de seu tempo em um laço estreito esperando que o dispositivo fique pronto.
- Essa abordagem é denominada espera ocupada.
- Se a CPU não tiver nada mais a fazer (por exemplo, a CPU de uma máquina de lavar), a espera ocupada pode servir (embora até um simples controlador muitas vezes precise monitorar vários eventos concorrentes).
- Todavia, se houver outro trabalho a fazer, como executar outros programas, a espera ocupada é um desperdício, portanto, é preciso um método de E/S diferente.

Figura 5.31 Exemplo de E/S programada.

```
public static void output_buffer(char buf[ ], int count) {
    // Produza um bloco de dados para o dispositivo
    int status, i, ready;
    for (i = 0; i < count; i++) {
        do {
            status = in(display_status_reg);      // obtenha estado
            ready = (status >> 7) & 0x01;         // isole o bit de pronto
        } while (ready != 1);
        out(display_buffer_reg, buf[i]);
    }
}
```

E/S por interrupção

Figura 5.30 Registradores de dispositivo para um terminal simples.

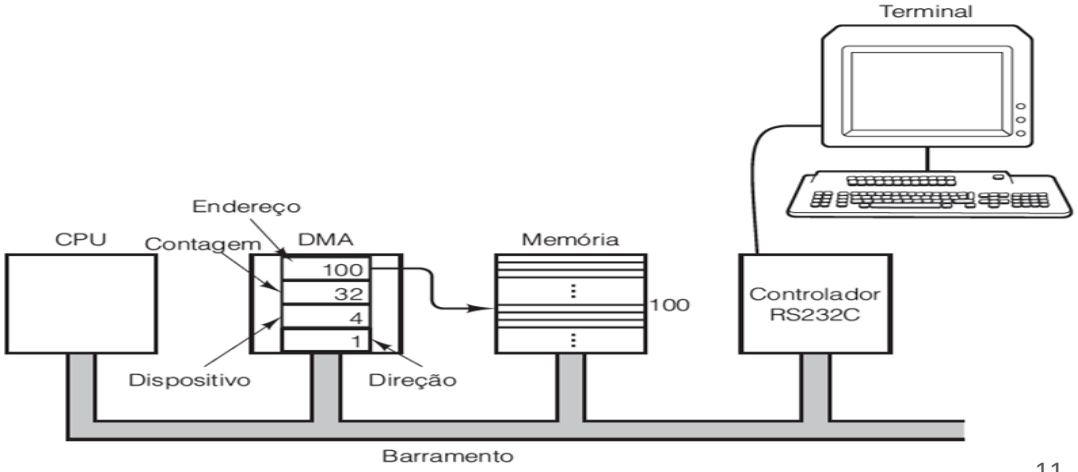


- O modo de se livrar da espera ocupada é fazer com que a CPU inicie o dispositivo de E/S e diga a ele para gerar uma interrupção quando concluir.
- Ajustando o bit INTERRUPT ENABLE (interrupção habilitada) em um registrador de dispositivo, o software pode requisitar que o hardware lhe dê um sinal quando a E/S for concluída.
- Embora a E/S por interrupção seja um grande passo à frente em comparação com a E/S programada, está longe de ser perfeita.
  - O problema é que é requerida uma interrupção para todo caractere transmitido.
  - Processar uma interrupção é caro, precisamos de um meio de nos livrar da maioria das interrupções.

E/S por DMA

- A solução está em voltar à E/S programada, mas contratar alguém para fazê-la.
- A Figura 5.32 mostra como isso é feito.
- Aqui, acrescentamos ao sistema um novo chip, um controlador DMA (Direct Memory Access – acesso direto à memória), com acesso direto ao barramento.

Figura 5.32 Sistema com um controlador DMA.

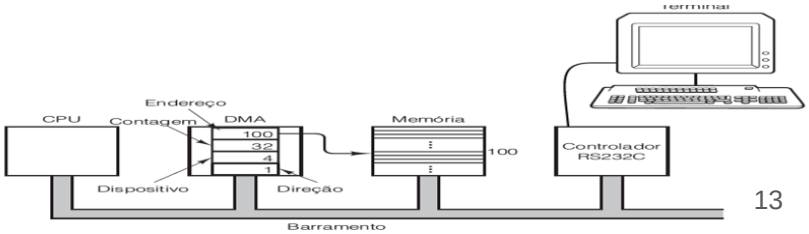


E/S por DMA

- O chip DMA contém, no mínimo, quatro registradores, e todos podem ser carregados por software executado na CPU.
  - O primeiro contém o endereço de memória a ser lido ou escrito.
  - O segundo contém a quantidade de bytes (ou palavras) a ser transferida.
  - O terceiro especifica o número do dispositivo ou espaço de endereço de E/S a utilizar, especificando, assim, qual é o dispositivo de E/S desejado.
  - O quarto informa se dados devem ser lidos ou escritos para o dispositivo de E/S.
- Para escrever um bloco de 32 bytes do endereço de memória 100 para um terminal (por exemplo, dispositivo 4), a CPU:
  - escreve os números 32, 100 e 4 nos três primeiros registradores DMA
  - e então o código para WRITE (por exemplo, 1) no quarto, como mostra a Figura.
- Uma vez inicializado desse modo, o controlador DMA faz uma requisição ao barramento para ler o byte 100 da memória, do mesmo modo que a CPU leria da memória.

E/S por DMA

- Quando a contagem por fim chega a 0, o controlador DMA para de transferir dados e ativa a linha de interrupção no chip da CPU.
  - Com DMA, a CPU tem de inicializar apenas alguns registradores.
- Logo após, ela está livre para fazer qualquer outra coisa até que a transferência completa seja concluída, quando então obtém uma interrupção do controlador DMA.
  - Alguns desses controladores têm dois, três ou mais conjuntos de registradores, portanto, podem controlar várias transferências simultâneas.



Introdução

Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).

Introdução

- Esse nível está posicionado entre o da microarquitetura e o do sistema operacional.
- Historicamente, ele foi desenvolvido antes de quaisquer outros níveis e, na verdade, na origem era o único.
- Até hoje, não é raro ouvir esse nível ser chamado de “a arquitetura” de uma máquina
- O nível ISA tem um significado especial que o torna importante para arquitetos de sistemas: **é a interface entre o software e o hardware**.

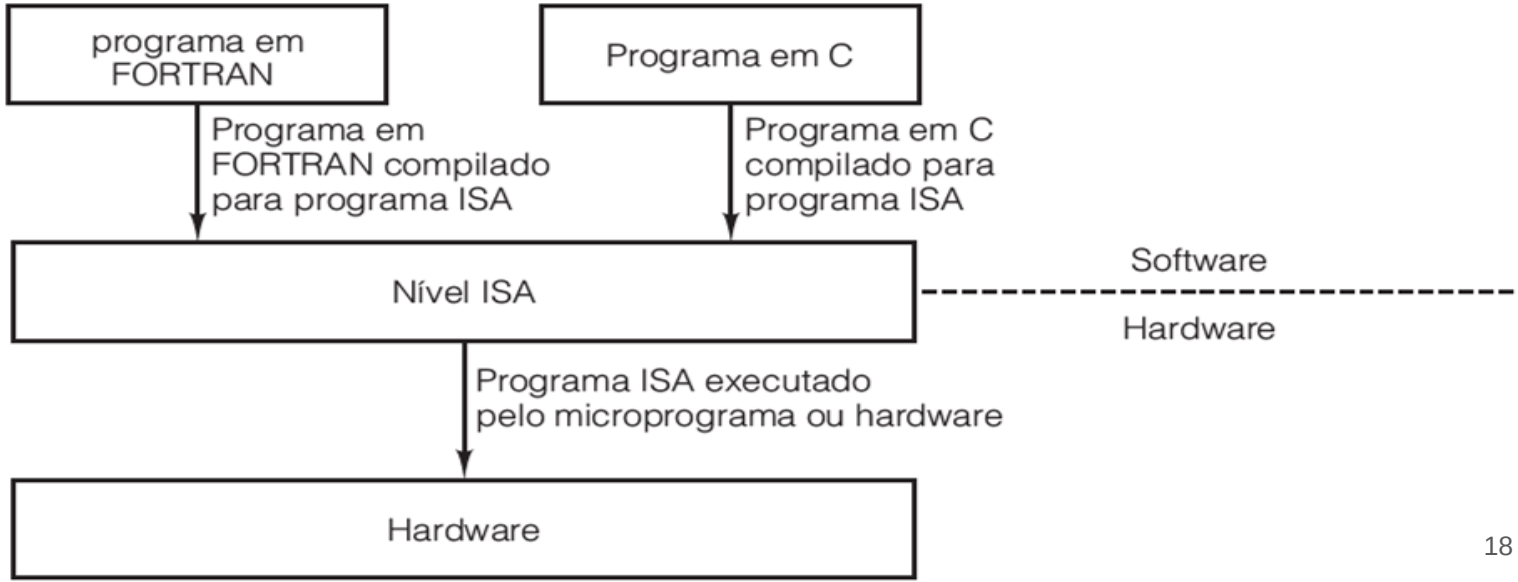
Introdução

- A abordagem adotada pelos projetistas é traduzir programas escritos em várias linguagens de alto nível para uma forma intermediária comum chamado nível ISA
- E construir hardware que possa executar programas diretamente no nível ISA.

Introdução

- O nível ISA define a interface entre os compiladores e o hardware.
  - É a linguagem que ambos têm de entender.
- A relação entre os compiladores, o nível ISA e o hardware pode ser vista na Figura 5.1.

Figura 5.1 O nível ISA é a interface entre os compiladores e o hardware.



Introdução

- **Na teoria o ideal é que**, ao projetar uma nova máquina, os arquitetos conversem com os escritores de compiladores e também com os engenheiros de hardware para descobrir quais características cada um deles quer no nível ISA.
- Após muita negociação e simulação, surgirá uma ISA que será implementada.

Introdução

- **Mas na realidade existem outras preocupações.**
- Quando surge uma nova máquina:
- A 1° pergunta que todos os clientes potenciais fazem é: “Ela é compatível com sua antecessora?”.
- A 2° é: “Ela pode executar meu sistema operacional antigo?”.
- A 3° é: “Ela executará todos os meus programas de aplicação existentes sem modificação?”.
- É raro que os clientes se disponham a jogar fora todos os seus programas antigos e começar tudo de novo.

Introdução

- Essa atitude pressiona muito os arquitetos de computadores a manter a mesma ISA entre modelos, ou torná-la compatível com os modelos anteriores.
- Com isso, a nova máquina deve ser capaz de executar programas antigos.
- Contudo, é totalmente aceitável que a nova máquina tenha novas instruções e outras características que só possam ser exploradas por novo software.

Introdução

- Nesse sentido, os projetistas têm toda a liberdade para fazer o que quiserem com o hardware,
  - podem adicionar paralelismo ou facilidades superescalares ou qualquer outra coisa que queiram, contanto que mantenham a compatibilidade com a ISA anterior.
- **Então, o desafio se torna construir máquinas melhores sujeitas às restrições de compatibilidade.**
- As ISAs podem ser responsáveis por uma diferença de até 25% em desempenho.

Introdução

- O que faz uma ISA ser boa?
- Primeiro, ela deve definir um conjunto de instruções que pode ser executado com eficiência em tecnologias atuais e futuras **(difícil)**
  - Resultando em projetos efetivos em custo por várias gerações.
- Segundo, ela deve fornecer um alvo claro para o código compilado.
  - Regularidade e completude de uma faixa de opções são aspectos importantes que nem sempre estão presentes em uma ISA.

Propriedades do nível ISA

- O código de nível ISA é o que um compilador produz (ignorando, por enquanto, chamadas ao sistema operacional).
- Para produzir código de nível ISA, o escritor de compilador tem de saber:
  - modelo de memória
  - quais e quantos são os registradores,
  - quais tipos de dados
  - quais instruções estão disponíveis
- O conjunto de todas essas informações define o nível ISA.

Propriedades do nível ISA

- Outra propriedade importante do nível ISA é que, na maioria das máquinas, há no mínimo dois modos.
  - **O modo núcleo (kernel) deve executar o sistema operacional e permite que todas as instruções sejam executadas.**
  - **O modo usuário (user) deve executar programas de aplicação e não permite que certas instruções sensíveis (como as que manipulam a cache diretamente) sejam executadas.**

Propriedades do nível ISA

- Para algumas arquiteturas, o nível ISA é especificado por um documento formal de definição.
- Por exemplo, a ARM v7 (ISA ARM versão 7) tem uma definição oficial publicada pela ARM Ltd.
- A finalidade de um documento de definição é possibilitar que diferentes fabricantes construam a máquina e todas elas executem exatamente o mesmo software e obtenham resultados idênticos.

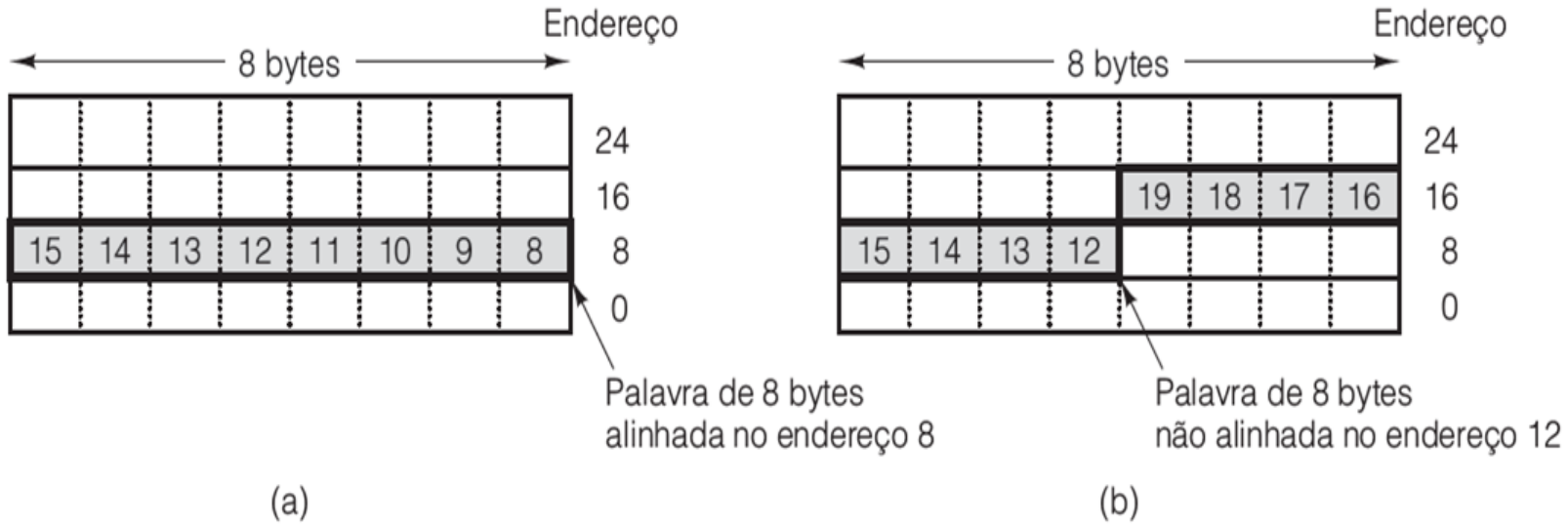
Modelos de memória

- Todos os computadores dividem a memória principal em células que têm endereços consecutivos.
- Muitas arquiteturas precisam que as palavras sejam alinhadas em seus limites naturais.
- Por exemplo, uma palavra de 4 bytes pode começar no endereço 0, 4, 8 etc., mas não no endereço 1 ou 2.
- De modo semelhante, uma palavra de 8 bytes pode começar no endereço 0, 8 ou 16, mas não no endereço 4 ou 6.

Modelos de memória

- O alinhamento de palavras de 8 bytes é ilustrado na Figura 5.2.

Figura 5.2 Palavra de 8 bytes em uma memória *little-endian*. (a) Alinhada. (b) Não alinhada. Algumas máquinas requerem que palavras na memória sejam alinhadas.



Modelos de memória

- O alinhamento costuma ser exigido porque memórias funcionam com mais eficiência desse modo.
- O Core i7, por exemplo, busca 8 bytes por vez na memória e admite apenas acessos alinhados em 64 bits.
- Assim, o Core i7 não poderia fazer uma referência desalinhada à memória nem que quisesse, porque a interface de memória exige endereços que sejam múltiplos de 8.

Modelos de memória

- O alinhamento costuma ser exigido porque memórias funcionam com mais eficiência desse modo.
- O Core i7, por exemplo, busca 8 bytes por vez na memória e admite apenas acessos alinhados em 64 bits.
- Assim, o Core i7 não poderia fazer uma referência desalinhada à memória nem que quisesse, porque a interface de memória exige endereços que sejam múltiplos de 8.

Modelos de memória

- Contudo, esse requisito de alinhamento às vezes causa problemas.
- O Core i7 permite que programas ISA referenciem palavras que começam em qualquer endereço.
- Se um programa Core i7 ler uma palavra de 4 bytes no endereço 7, o hardware tem de fazer uma referência à memória para obter os bytes de 0 a 7 e uma segunda referência à memória para obter os bytes de 8 a 15.
- Então, a CPU tem de extrair os 4 bytes requisitados dos 16 bytes lidos da memória e montá-los na ordem correta para formar uma palavra de 4 bytes.
- Fazer isso regularmente não leva a uma velocidade fantástica.
  - Compatibilidade!

Modelos de memória

- Algumas máquinas têm espaços de endereços separados para **instruções e dados**
- Uma busca de instrução no endereço 8 vai para um espaço de endereço diferente de uma busca de dados no endereço 8.
- Com as instruções e dados separados, dependendo da leitura ser de instrução ou de dados, terá resultados diferentes.

Registradores

- Todos os computadores têm alguns registradores visíveis no nível ISA.
- Eles estão lá para controlar a execução do programa, reter resultados temporários e para outras finalidades.
- Os registradores visíveis no nível de microarquitetura, como MAR (*Memory Address Register*), não o são no nível ISA.
- Alguns, como o contador de programa e o ponteiro de pilha, são visíveis em ambos os níveis.
- Registradores visíveis no nível ISA são sempre visíveis no nível da microarquitetura, já que é ali que são implementados.

Registradores

- Registradores de nível ISA podem ser divididos em duas categorias: **de uso especial e de uso geral.**
- Os de uso especial incluem o contador de programa e o ponteiro de pilha, bem como outros registradores com uma função específica.
- Os registradores de uso geral contêm as variáveis locais fundamentais e resultados intermediários de cálculos.
  - Sua função principal é prover acesso rápido a dados muito usados (basicamente evitando acessos à memória).
  - Máquinas RISC costumam ter ao menos 32 registradores de uso geral.

Registradores

- Se os registradores de uso geral forem todos equivalentes, um compilador pode usar R1 para reter um resultado temporário, mas também pode usar R25 da mesma forma.
- A escolha de registrador não importa
- Uso de uma convenção é recomendado

Registradores

- Além dos registradores de nível ISA visíveis aos programas do usuário, há sempre uma grande quantidade de registradores de uso especial, disponíveis somente em modo núcleo.
  - Eles controlam as várias caches, memória, dispositivos de E/S e outros recursos de hardware da máquina.
  - São usados apenas pelo sistema operacional, de modo que compiladores e usuários não precisam conhecê-los.

Registradores

- Um registrador de controle, que é algo como um híbrido de núcleo/usuário, é o registrador de flags ou **PSW** (Program Status Word – palavra de estado do programa).
- Esse registrador contém vários bits, necessários para CPU.
- Os mais importantes são os códigos de condição.
- Esse código contém bits, que são ajustados a cada ciclo da ULA e refletem o estado do resultado da operação mais recente.

Registradores

- Entre os bits de condição típicos estão:

  N – Marcado quando o resultado foi Negativo.

  Z – Marcado quando o resultado foi Zero.

  V – Marcado quando o resultado causou um transbordo (overflow).

  C – Marcado quando o resultado causou um vai-um (Carry) do bit da extrema esquerda.

  P – Marcado quando o resultado teve Paridade par.

Registradores

- Os códigos de condição são importantes porque as instruções de comparação e desvio condicional os utilizam.
- Por exemplo, a instrução CMP (comparação) normalmente subtrai dois operandos e ajusta os códigos de condição com base na diferença.
- Se os operandos forem iguais, então a diferença será zero, e o bit de código de condição Z no registrador PSW será marcado.
- Uma instrução BEQ (Branch EQual) testa o bit Z e desvia se ele estiver marcado.
  - Instrução utilizada quando desejamos comparar a igualdade de dois valores armazenados em registradores.
  - Sintaxe:
  - beq $r1, $r2, L1,
  - Onde $r1, $r2 são os registradores cujos valores armazenados vão ser comparados.
  - Se os valores são iguais, a sequência de execução pula para a instrução que possui o rótulo L1.

Instruções

- A principal característica do nível ISA é o seu conjunto de instruções de máquina, que controlam o que a máquina pode fazer.
- Há sempre:
- Instruções LOAD e STORE para mover dados entre a memória e registradores.
- Instruções MOVE para copiar dados entre os registradores.
- Instruções aritméticas,
- Instruções booleanas e
- Instruções para comparar dados e desviar conforme os resultados.

Tipos de dados

- Todos os computadores precisam de dados.
- Os dados têm de ser representados de alguma forma específica no interior do computador.
- No nível ISA, são usados vários tipos de dados diferentes.
- Uma questão fundamental é se há ou não suporte de hardware para um tipo particular de dados.

Tipos de dados

- Suporte de hardware significa que uma ou mais instruções esperam dados em um formato particular e o usuário não tem liberdade de escolher um diferente.
  - Por exemplo, é comum escrever números negativos com um sinal de menos à esquerda do número em vez de à direita.
- O hardware espera certo formato para inteiros, por exemplo, e não funciona direito quando recebe qualquer outra coisa.

Atividade em Sala

- Quais as diferenças entre processadores CISC e RISC?
- Quais as vantagens e desvantagens de cada uma das arquiteturas?

Atividade para Casa

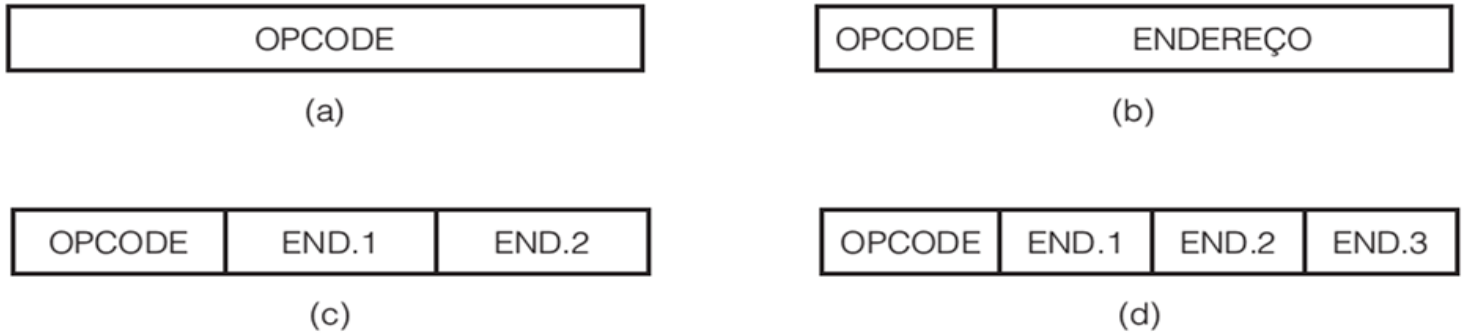
- Levando em consideração as seguintes arquiteturas (Intel x86, x64, ARM e AVR)

- Quais os principais registradores de cada uma das arquiteturas?
- Qual o modelo de memória?
- Quais tipos de dados são suportados?
- Quais são as principais instruções?

Formatos de instrução

- Uma instrução consiste em um opcode, normalmente em conjunto com alguma informação adicional, tais como de onde vêm os operandos e para onde vão os resultados.
- A Figura 5.9 mostra diversos formatos possíveis para instruções de nível 2. Instruções sempre têm um opcode que indica o que a instrução faz.
- Pode haver zero, um, dois ou três endereços presentes.

Figura 5.9 Quatro formatos comuns de instrução: (a) Instrução sem endereço. (b) Instrução de um endereço. (c) Instrução de dois endereços. (d) Instrução de três endereços.



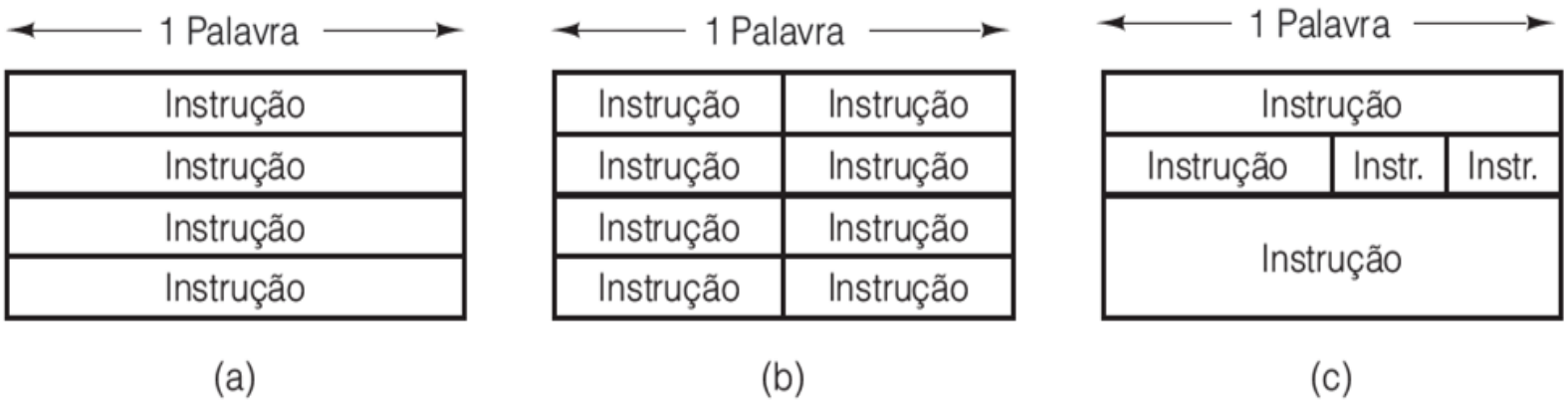
Formatos de instrução

- Em algumas máquinas, todas as instruções têm o mesmo comprimento; em outras, pode haver muitos comprimentos diferentes.
- Instruções podem ser mais curtas, mais longas ou do mesmo comprimento da palavra.
- Ter instruções do mesmo comprimento da palavra é mais simples e facilita a decodificação, mas muitas vezes desperdiça espaço, já que, desse modo, todas as instruções precisam ser tão longas quanto a mais longa.

Formatos de instrução

- A Figura 5.10 mostra algumas relações possíveis entre comprimento de instrução e comprimento de palavra.

Figura 5.10 Algumas relações possíveis entre comprimento de instrução e de palavra.



Endereçamento

- Grande parte das instruções tem operandos, portanto, é necessário algum modo de especificar onde eles estão.
- A isso nos referimos por endereçamento.

Endereçamento imediato

- O modo mais simples de uma instrução especificar um operando é o endereço conter o operando de fato em vez de um endereço ou outra informação que descreva onde ele está.
- Tal operando é denominado **operando imediato** porque ele é automaticamente buscado na memória, ao mesmo tempo em que a própria instrução;
- Essa forma, ele está de pronto disponível para uso.

Endereçamento imediato

- Uma possível instrução imediata para carregar o registrador R1 com a constante 4 é mostrada na Figura 5.16.

Figura 5.16 Instrução imediata para carregar 4 no registrador 1.

MOV	R1	4

Endereçamento imediato

- O endereçamento imediato tem a vantagem de não exigir uma referência extra à memória para buscar o operando.
- A desvantagem é que só uma constante pode ser fornecida desse modo.
- Além disso, o número de valores é limitado pelo tamanho do campo.
- Ainda assim, muitas arquiteturas usam essa técnica para especificar constantes inteiras pequenas.

Endereçamento direto

MOV	R1	#4

- Um método para especificar um operando na memória é dar seu endereço completo.
- Esse modo é denominado **endereçamento direto**.
- Assim como o imediato, o endereçamento direto tem uso restrito: **a instrução sempre acessará exatamente a mesma localização de memória.**
- Portanto, embora o valor possa mudar, sua localização não pode.
- Assim, **o endereçamento direto só pode ser usado para acessar variáveis globais** cujos endereços sejam conhecidos no momento da compilação.
- Muitos programas têm variáveis globais, portanto, esse modo é muito utilizado.

Endereçamento de registrador

CMP R2,R3

- **Endereçamento de registrador** é conceitualmente o mesmo que endereçamento direto, **mas especifica um registrador em vez de uma localização de memória.**
- Como os registradores são tão importantes (pelo acesso rápido e endereços curtos), esse modo de endereçamento é o mais comum na maioria dos computadores.
- Muitos compiladores fazem todo o possível para determinar quais variáveis serão acessadas com maior frequência (por exemplo, o índice de um laço) e as colocam em registradores.

Endereçamento indireto de registrador (ponteiro)

- Nesse modo, **o operando que está sendo especificado vem da memória ou vai para ela**, mas seu endereço não está ligado à instrução, como no endereçamento direto. **Em vez disso, está contido em um registrador.**
- Quando um endereço é usado dessa maneira, ele é denominado **ponteiro**.
- Uma grande vantagem do endereçamento indireto de registrador é que ele **pode referenciar a memória sem pagar o preço de ter um endereço de memória completo na instrução.**
- Além disso, também pode usar diferentes palavras de memória em diferentes execuções da instrução.

```
ADD R1,(R2)
```

Endereçamento indexado

- Muitas vezes, é útil poder referenciar palavras de memória cujo deslocamento em relação a um registrador é conhecido.
- Endereçamento indexado é o nome que se dá ao endereçamento de memória que fornece um registrador (explícito ou implícito) mais um deslocamento constante.

```
AND R4,B(R2)
```

Tipos de instrução

- Instruções de nível ISA podem ser divididas em grupos que guardam relativa semelhança de uma máquina para outra, ainda que possam ser diferentes nos detalhes.
- Alguns exemplos:
  - Load
  - Store
  - Move
  - Incremento
  - Negação (Not)
  - Desempilhar
  - ...

Instruções para movimento de dados

- Copiar dados de um lugar para outro é a mais fundamental de todas as operações.
  - **LOAD, STORE e MOVE**
- Por copiar queremos dizer a criação de um novo objeto, com padrão de bits idêntico ao original.
- Esse uso da palavra “movimento”, ou “mudança”, é um pouco diferente de sua utilização em linguagem corrente.
- Quando dizemos que o conteúdo do local de memória 2000 “foi movido” para algum registrador, isso sempre significa que uma cópia idêntica foi criada no registrador e que o original continua intocável no local 2000.

Operações diádicas

- **Operações diádicas são as que combinam dois operandos para produzir um resultado.**
  - EX: instruções para efetuar adição e subtração de inteiros.
  - EX: as instruções booleanas.

Operações monádicas

- **Operações monádicas têm um só operando e produzem um só resultado.**
- EX: Instruções para deslocar ou rodar o conteúdo de uma palavra ou byte.
- Deslocamentos são operações nas quais os bits são movidos para a esquerda ou para a direita, com bits que são empurrados para fora na extremidade da palavra e, **portanto, perdidos.**
- Rotações são deslocamentos nos quais os bits empurrados para fora por uma extremidade reaparecem na outra.

Operações monádicas

- Deslocamentos e rotações para a esquerda e para a direita são ambos úteis.
- Se uma palavra de n bits for rodada k bits para a esquerda, o resultado será o mesmo caso ela tivesse sido rodada n – k bits para a direita.
- Deslocamentos para a direita costumam ser executados com extensão de sinal, o que significa que posições desocupadas na extremidade esquerda da palavra são preenchidas com o bit de sinal original, 0 (positivo) ou 1 (negativo).

00000000 00000000 00000000 01110011	A
00000000 00000000 00000000 00011100	A deslocado 2 bits para a direita
11000000 00000000 00000000 00011100	A rodado 2 bits para a direita

Operações monádicas - PARA CASA

- Explique de maneira prática como o deslocamento ou rotação é importante para o nosso computador.
- Você acha que conseguimos fazer multiplicação ou divisão por meio de deslocamento? Se sim, explique.
- Será que conseguimos empacotar ou desempacotar sequências de bits usando a rotação? Explique.

Comparações e desvios condicionais

- Programas precisam da capacidade de testar seus dados e alterar a sequência de instruções a ser executada com base nos resultados.
  - Um exemplo é a função raiz quadrada, x. Se x for negativo, o procedimento emite uma mensagem de erro; caso contrário, calcula a raiz quadrada.
  - Uma função sqrt tem de testar x e em seguida desviar, dependendo de ele ser negativo ou não.
- Um método comum para fazer isso é providenciar instruções de desvio condicional que testam alguma condição e desviam para um determinado endereço de memória se a condição for cumprida.
- Às vezes, um bit na instrução indica se o desvio deve ocorrer se a condição for cumprida ou se a condição não for cumprida.

Comparações e desvios condicionais

- Comparar duas palavras ou caracteres para ver se são iguais, ou, se não forem, qual deles é maior, também é importante, por exemplo, na classificação.
- Para executar esse teste, são necessários três endereços:
  - dois para os itens de dados
  - e um para o endereço para onde desviar se a condição for verdadeira.

```
CMP R2,R3        ; já terminamos?
BLT LOOP         ; se R2 < R3, não terminamos, portanto, continue
```

Comparações e desvios condicionais

Outro ponto sutil em relação à comparação de números é decidir se os números devem ser ou não considerados com sinal.

Números binários de três bits podem ser ordenados conforme um de dois modos.

Do menor para o maior:

Sem sinal	Com sinal	
000	100	(o menor)
001	101	
010	110	
011	111	
100	000	
101	001	
110	010	
111	011	(o maior)

Comparações e desvios condicionais

- A coluna da esquerda mostra os inteiros positivos de 0 a 7 em ordem crescente. A coluna da direita mostra os inteiros com sinal de –4 a +3 em complemento de dois. A resposta à pergunta “011 é maior do que 100?”

Comparações e desvios condicionais

- A coluna da esquerda mostra os inteiros positivos de 0 a 7 em ordem crescente. A coluna da direita mostra os inteiros com sinal de –4 a +3 em complemento de dois. A resposta à pergunta “011 é maior do que 100?”

  Depende de considerar ou não que os números têm sinal. A maioria das ISAs tem instruções para tratar ambas as ordenações.

Instruções de chamada de procedimento

- Um procedimento é um grupo de instruções que realiza alguma tarefa e pode ser invocado (chamado) de diversas partes do programa.
- O termo sub-rotina muitas vezes é usado em vez de procedimento, em especial quando se refere a programas em linguagem de montagem.
- Em C, procedimentos são denominados funções, embora não sejam necessariamente funções no sentido matemático.
- Em Java, o termo usado é método.

Instruções de chamada de procedimento

- Quando o procedimento conclui sua tarefa, deve retornar à declaração após a chamada.
- Portanto, o endereço de retorno deve ser transmitido ao procedimento ou salvo em algum lugar de modo que possa ser localizado quando for hora de retornar.
- O endereço de retorno pode ser colocado em qualquer um de três lugares:
  - na memória,
  - em um registrador
  - **ou na pilha**.

Controle de laço

- (a) Laço do tipo “teste no final” e (b) Laço do tipo “teste no início”.

```
        i = 1;
L1:     primeira declaração;
        .
        .
        .
        última declaração;
        i = i + 1;
        if (i < n) goto L1;
```

(a)

```
        i = 1;
L1:     if (i > n) goto L2;
        primeira declaração;
        .
        .
        .
        última declaração;
        i = i + 1;
        goto L1;
L2:
```

(b)

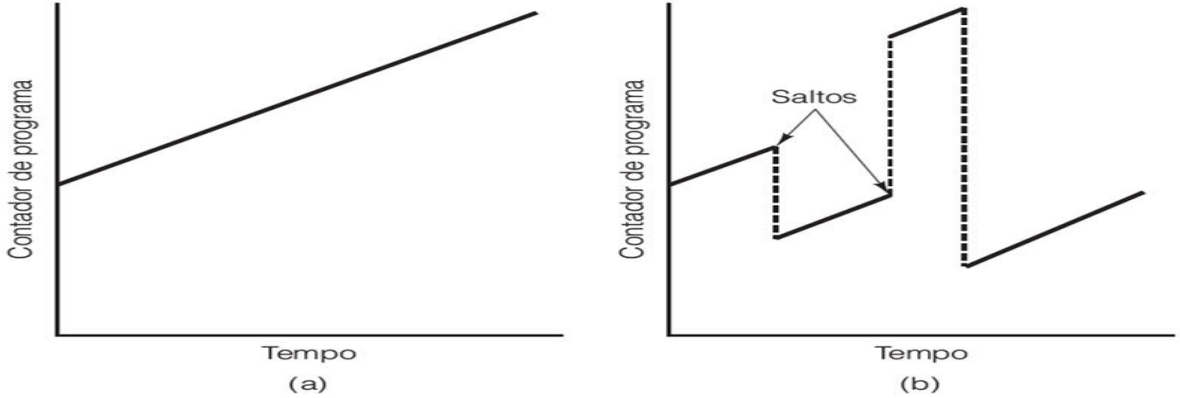
Fluxo de controle

- Fluxo de controle se refere à sequência em que as instruções são executadas dinamicamente, isto é, durante a execução do programa.
- Em geral, na ausência de desvios e chamadas de procedimento, instruções executadas em sequência são buscadas em locais consecutivos de memória.
- Chamadas de procedimento causam alteração no fluxo de controle, interrompendo o procedimento que está sendo executado naquele momento e iniciando o procedimento chamado.

Fluxo de controle sequencial e desvios

- A maioria das instruções não altera o fluxo de controle.
  - Após uma instrução ser executada, a que vem depois dela na memória é buscada e executada.
  - Após cada instrução, o contador de programa é aumentado pelo comprimento da instrução.
- Se isso for observado por um intervalo de tempo longo em comparação com o tempo médio de instrução, o contador de programa é mais ou menos uma função linear do tempo
- Se um programa contém desvios, essa relação já não vale mais.

Figura 5.36 Contador de programa como função de tempo (ajustada). (a) Sem desvios. (b) Com desvios.



Exceções

- Uma exceção (trap) é um tipo de chamada de procedimento automática iniciada por alguma condição causada pelo programa.
- Um bom exemplo é o transbordo (overflow).
- Em muitos computadores, se o resultado de uma operação aritmética for maior do que o maior número que pode ser representado, ocorre uma exceção,
- O fluxo de controle passa para algum local fixo de memória em vez de continuar em sequência.
- Nesse local fixo há um desvio para uma chamada de procedimento denominada tratador de exceção que executa alguma ação adequada, como imprimir uma mensagem de erro.

Exceções

- O ponto essencial de uma exceção é que ela é iniciada por alguma condição excepcional causada pelo próprio programa e detectada pelo hardware ou microprograma.
- Algumas das condições comuns que podem causar exceções são:
  - transbordo ou erro de ponto flutuante,
  - transbordo de inteiros,
  - violação de proteção,
  - opcode indefinido,
  - transbordo de pilha,
  - tentativa de iniciar dispositivo de E/S inexistente,
  - tentativa de buscar uma palavra em endereço de número ímpar
  - e divisão por zero.

Tarefa para Sala

- Quais as diferenças entre interrupções e exceções?